ML Lab-09 Aim:To implement the K-means clustering algorithm and analyze the clustering results using appropriate evaluation techniques.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
path = '/content/drive/MyDrive/dataset_9/Mall_Customers.csv'
df = pd.read_csv(path)
df.head(5)
```

| | CustomerID | Gender | Age | Annual Income (k$) | Spending Score (1-100) |
|---|---|---|---|---|---|
| **0** | 1 | Male | 19 | 15 | 39 |
| **1** | 2 | Male | 21 | 15 | 81 |
| **2** | 3 | Female | 20 | 16 | 6 |
| **3** | 4 | Female | 23 | 16 | 77 |
| **4** | 5 | Female | 31 | 17 | 40 |

Next steps: Generate code with df New interactive sheet

```
df.shape
```

```
(200, 5)
```

```
df.columns
```

```
Index(['CustomerID', 'Gender', 'Age', 'Annual Income (k$)',
       'Spending Score (1-100)'],
      dtype='object')
```

```
df.isnull().any().any()
```

```
np.False_
```

```
X = df[['Annual Income (k$)', 'Spending Score (1-100)']]
```

```
X.head()
```

| | Annual Income (k$) | Spending Score (1-100) |
|---|---|---|
| **0** | 15 | 39 |
| **1** | 15 | 81 |
| **2** | 16 | 6 |
| **3** | 16 | 77 |
| **4** | 17 | 40 |

Next steps: Generate code with X New interactive sheet

```
X = pd.get_dummies(X,dtype = int)
```

```
X.head()
```

| | Annual Income (k$) | Spending Score (1-100) |
|---|---|---|
| **0** | 15 | 39 |
| **1** | 15 | 81 |
| **2** | 16 | 6 |
| **3** | 16 | 77 |
| **4** | 17 | 40 |

Next steps: Generate code with X New interactive sheet

```
x =df.iloc[:, [3,4]].values
print(X)
```

```
     Annual Income (k$)  Spending Score (1-100)
0                    15                      39
1                    15                      81
2                    16                       6
3                    16                      77
4                    17                      40
..                  ...                     ...
195                 120                      79
196                 126                      28
197                 126                      74
198                 137                      18
199                 137                      83

[200 rows x 2 columns]
```

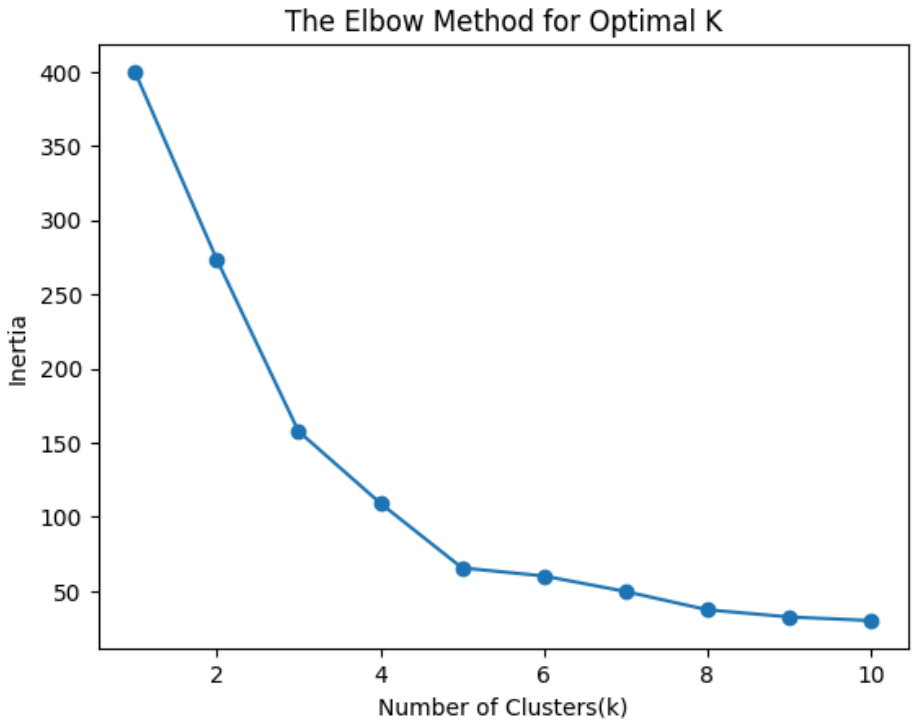
```
scaler = StandardScaler()
x_scaled = scaler.fit_transform(x)
```

```
print(x_scaled)
```

```
 [ 0.66567484  1.54509812]
 [ 0.66567484 -1.28887582]
 [ 0.66567484  1.46745499]
 [ 0.66567484 -1.17241113]
 [ 0.66567484  1.00159627]
 [ 0.66567484 -1.32769738]
 [ 0.66567484  1.50627656]
 [ 0.66567484 -1.91002079]
 [ 0.66567484  1.07923939]
 [ 0.66567484 -1.91002079]
 [ 0.66567484  0.88513158]
 [ 0.70384427 -0.59008772]
 [ 0.70384427  1.27334719]
 [ 0.78018313 -1.75473454]
 [ 0.78018313  1.6615628 ]
 [ 0.93286085 -0.93948177]
 [ 0.93286085  0.96277471]
 [ 0.97103028 -1.17241113]
 [ 0.97103028  1.73920592]
 [ 1.00919971 -0.90066021]
 [ 1.00919971  0.49691598]
 [ 1.00919971 -1.44416206]
 [ 1.00919971  0.96277471]
 [ 1.00919971 -1.56062674]
 [ 1.00919971  1.62274124]
 [ 1.04736914 -1.44416206]
 [ 1.04736914  1.38981187]
 [ 1.04736914 -1.36651894]
 [ 1.04736914  0.72984534]
 [ 1.23821628 -1.4053405 ]
 [ 1.23821628  1.54509812]
 [ 1.390894   -0.7065524 ]
 [ 1.390894    1.38981187]
 [ 1.42906343 -1.36651894]
 [ 1.42906343  1.46745499]
 [ 1.46723286 -0.43480148]
 [ 1.46723286  1.81684904]
 [ 1.54357172 -1.01712489]
 [ 1.54357172  0.69102378]
 [ 1.61991057 -1.28887582]
 [ 1.61991057  1.35099031]
 [ 1.61991057 -1.05594645]
 [ 1.61991057  0.72984534]
 [ 2.00160487 -1.63826986]
 [ 2.00160487  1.58391968]
 [ 2.26879087 -1.32769738]
 [ 2.26879087  1.11806095]
 [ 2.49780745 -0.86183865]
 [ 2.49780745  0.92395314]
 [ 2.91767117 -1.25005425]
 [ 2.91767117  1.27334719]]
```

```
inertia_values = []
for k in range(1,11):
  kmeans = KMeans(n_clusters=k,random_state=42)
  kmeans.fit(x_scaled)
  inertia_values.append(kmeans.inertia_)
print(inertia_values)
plt.figure()
plt.plot(range(1,11),inertia_values,marker='o')
plt.xlabel('Number of Clusters(k)')
plt.ylabel('Inertia')
plt.title('The Elbow Method for Optimal K')
plt.show()
```

```
[399.99999999999994, 273.66888662642003, 157.70400815035939, 109.22822707921345, 65.56840815571681, 60
```




```
Kmeans = KMeans(n_clusters=9,random_state=42)
clusters = Kmeans.fit_predict(x_scaled)
df['clusters'] = clusters
print(df.head())
```

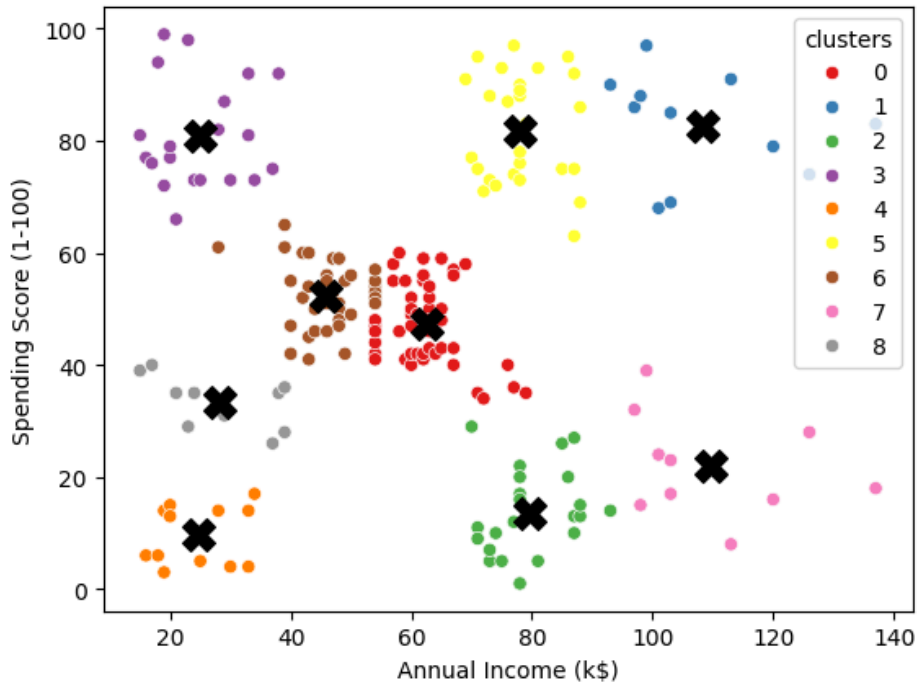
```
   CustomerID  Gender  ...  Spending Score (1-100)  clusters
0           1    Male  ...                      39         8
1           2    Male  ...                      81         3
2           3  Female  ...                       6         4
3           4  Female  ...                      77         3
4           5  Female  ...                      40         8

[5 rows x 6 columns]
```

```
print("Intertia:",Kmeans.inertia_)
sil_Score = silhouette_score(x_scaled,clusters)
print("Silhouette Score:",sil_Score)
```

```
Intertia: 32.495081199100916
Silhouette Score: 0.4566624374485964
```

```
plt.figure
sns.scatterplot(x='Annual Income (k$)',y='Spending Score (1-100)',hue='clusters',data=df,palette='Set1
centroids = scaler.inverse_transform(Kmeans.cluster_centers_)
plt.scatter(centroids[:,0],centroids[:,1],marker='X',s=200,c='black',label='Centroids')
```

```
<matplotlib.collections.PathCollection at 0x78ded92b9250>
```




Start coding or generate with AI.